# Exercício: Criando uma Interface de Veículo

### Objetivo

O objetivo deste exercício é praticar a implementação de interfaces em Java e o uso de métodos padrão (`default`) em interfaces.

### Instruções

1. Crie uma interface chamada `Veiculo` com os métodos `iniciar`, `parar` e um método padrão `buzinar`.
2. Implemente a interface `Veiculo` em duas classes: `Carro` e `Caminhao`.
3. Crie uma classe chamada `Main` com o método `main` para testar as implementações das classes `Carro` e `Caminhao`.
4. Compile e execute o programa para verificar os resultados.

### Passos para Fazer o Exercício

1. **Criação da Interface `Veiculo`:**
   - Defina a interface `Veiculo` com os métodos `iniciar` e `parar`.
   - Adicione um método padrão `buzinar` que imprime "Buzinando" no console.
2. **Implementação da Interface `Veiculo`:**
   - Crie a classe `Carro` que implementa a interface `Veiculo`.
   - Implemente os métodos `iniciar` e `parar` na classe `Carro`.
   - Crie a classe `Caminhao` que implementa a interface `Veiculo`.
   - Implemente os métodos `iniciar` e `parar` na classe `Caminhao`.
3. **Criação da Classe `Main`:**
   - Defina a classe `Main`.
   - No método `main`, crie instâncias de `Carro` e `Caminhao`.
   - Chame os métodos `iniciar` e `buzinar` para cada instância.
   - Imprima os resultados no console.
4. **Compilação e Execução:**
   - Compile o código utilizando um compilador Java.
   - Execute o programa e verifique os resultados impressos no console.

### Código do Exercício

```java
interface Veiculo {
    void iniciar();
    void parar();
    default void buzinar() {
        System.out.println("Buzinando");
    }
}

class Carro implements Veiculo {
    public void iniciar() {
        System.out.println("Ligando o motor do carro...");
    }
    public void parar() {
        System.out.println("Parando o motor do carro...");
    }
}

class Caminhao implements Veiculo {
    public void iniciar() {
        System.out.println("Ligando o motor do caminhão...");
    }
    public void parar() {
        System.out.println("Parando o motor do caminhão...");
    }
}

public class Main {
    public static void main(String[] args) {
        Veiculo tesla = new Carro();
        Veiculo tata = new Caminhao();

        tesla.iniciar(); // Ligando o motor do carro...
        tesla.buzinar(); // Buzinando
        tata.iniciar(); // Ligando o motor do caminhão...
        tata.buzinar(); // Buzinando
    }
}
```

### Explicação do Código

- **Interface `Veiculo`:**
  - Define os métodos `iniciar` e `parar` que devem ser implementados pelas classes que a utilizam.
  - O método padrão `buzinar` imprime "Buzinando" no console, fornecendo uma implementação padrão que pode ser utilizada ou sobrescrita pelas classes que implementam a interface.
- **Classe `Carro`:**
  - Implementa a interface `Veiculo` e fornece as implementações dos métodos `iniciar` e `parar`.
- **Classe `Caminhao`:**
  - Também implementa a interface `Veiculo` e fornece suas próprias implementações dos métodos `iniciar` e `parar`.
- **Classe `Main`:**
  - No método `main`, são criadas instâncias de `Carro` e `Caminhao`.
  - Os métodos `iniciar` e `buzinar` são chamados para cada instância, demonstrando o uso da interface e do método padrão.

Este exercício ajuda a entender como implementar interfaces em Java e como utilizar métodos padrão em interfaces.

# Exercício: Interface para Pedidos de Restaurante

## Objetivo:

Criar uma interface chamada `PedidoRestaurante` que define métodos para adicionar itens ao pedido e calcular o valor total.

## Instruções:

1. Crie a interface `PedidoRestaurante` com os seguintes métodos:
   - `adicionarItem(String item, double preco)`: Adiciona um item ao pedido com o nome e preço.
   - `calcularTotal()`: Calcula o valor total do pedido.
2. Implemente a interface em uma classe chamada `Pedido` que mantém uma lista de itens e seus preços.
3. Teste sua implementação criando um pedido com alguns itens (por exemplo, "Hambúrguer", "Batata Frita" e "Refrigerante") e exiba o valor total.

## Passos para fazer o exercício:

1. Crie um novo projeto Java no seu ambiente de desenvolvimento.
2. Crie a interface `PedidoRestaurante` com os métodos especificados.
3. Implemente a classe `Pedido` que implementa a interface e mantém uma lista de itens e preços.
4. No método `main`, crie uma instância de `Pedido`, adicione alguns itens e exiba o valor total.

## Código do Exercício:

```java
interface PedidoRestaurante {
    void adicionarItem(String item, double preco);
    double calcularTotal();
}

public class Pedido implements PedidoRestaurante {
    private double total = 0;

    @Override
    public void adicionarItem(String item, double preco) {
        System.out.println("Adicionando " + item + " (R$" + preco + ") ao pedido.");
        total += preco;
    }

    @Override
    public double calcularTotal() {
        return total;
    }

    public static void main(String[] args) {
        PedidoRestaurante pedido = new Pedido();
        pedido.adicionarItem("Hambúrguer", 15.0);
        pedido.adicionarItem("Batata Frita", 8.0);
        pedido.adicionarItem("Refrigerante", 4.0);

        System.out.println("Valor total do pedido: R$" + pedido.calcularTotal());
    }
}
```

### Explicação do Código:

- A interface `PedidoRestaurante` define os métodos necessários para gerenciar um pedido de restaurante.
- A classe `Pedido` implementa essa interface e mantém um total acumulado dos preços dos itens.
- No método `main`, criamos um pedido, adicionamos alguns itens e exibimos o valor total.

# Exercício Complementar: Controle de Estoque

## Objetivo:

Praticar o uso de interfaces em Java para definir comportamentos e implementar uma classe capaz de controlar o estoque de um produto, permitindo consultar informações e atualizar quantidades disponíveis.

## Instruções:

1. Crie uma interface chamada `Produto` com os seguintes atributos:
   - `nome` (String): o nome do produto.
   - `quantidade` (int): a quantidade disponível em estoque.
2. Implemente uma classe chamada `ProdutoImpl` que implementa a interface `Produto`.
3. Na classe `ProdutoImpl`, crie um construtor que inicialize os atributos `nome` e `quantidade`.
4. Adicione métodos para:
   - Atualizar a quantidade de produtos em estoque (por exemplo, `adicionarQuantidade(int quantidade)` e `removerQuantidade(int quantidade)`).
   - Exibir informações sobre o produto (por exemplo, `getNome()`, `getQuantidade()`).

### Detalhes adicionais:

**Método `adicionarQuantidade(int quantidade)`:**

- Esse método deve receber um valor inteiro (`quantidade`) como parâmetro.
- Ele deve adicionar a quantidade especificada ao estoque atual do produto.
- Por exemplo, se o estoque atual for 50 e chamarmos `adicionarQuantidade(20)`, o novo estoque será 70.

**Método `removerQuantidade(int quantidade)`:**

- Similar ao método anterior, esse método também recebe um valor inteiro (`quantidade`) como parâmetro.
- Ele deve subtrair a quantidade especificada do estoque atual do produto.
- No entanto, antes de fazer a subtração, verifique se a quantidade em estoque é suficiente para a remoção. Se não for, exiba uma mensagem informando que a quantidade é insuficiente.
- Por exemplo, se o estoque atual for 70 e chamarmos `removerQuantidade(15)`, o novo estoque será 55. Se chamarmos `removerQuantidade(80)`, a mensagem de quantidade insuficiente deve ser exibida.

5. Crie um programa principal que demonstre o uso da classe `ProdutoImpl`.

### Código do exercício:

```java
interface Produto {
    String getNome();
    int getQuantidade();
    void adicionarQuantidade(int quantidade);
    void removerQuantidade(int quantidade);
}

class ProdutoImpl implements Produto {
    private String nome;
    private int quantidade;

    public ProdutoImpl(String nome, int quantidade) {
        this.nome = nome;
        this.quantidade = quantidade;
    }

    @Override
    public String getNome() {
        return nome;
    }

    @Override
    public int getQuantidade() {
        return quantidade;
    }

    @Override
    public void adicionarQuantidade(int quantidade) {
        this.quantidade += quantidade;
    }

    @Override
    public void removerQuantidade(int quantidade) {
        if (this.quantidade >= quantidade) {
            this.quantidade -= quantidade;
        } else {
            System.out.println("Quantidade insuficiente em estoque.");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Produto produto = new ProdutoImpl("Caneta", 50);
        System.out.println("Produto: " + produto.getNome());
        System.out.println("Quantidade em estoque: " + produto.getQuantidade());

        produto.adicionarQuantidade(20);
        System.out.println("Nova quantidade após adição: " + produto.getQuantidade());

        produto.removerQuantidade(15);
        System.out.println("Nova quantidade após remoção: " + produto.getQuantidade());
    }
}
```

### Explicação do código:

- A interface `Produto` define os métodos `getNome()`, `getQuantidade()`, `adicionarQuantidade(int quantidade)` e `removerQuantidade(int quantidade)`.
- A classe `ProdutoImpl` implementa a interface `Produto` e fornece a lógica para atualizar a quantidade de produtos em estoque.
- O programa principal (`Main`) cria um objeto `ProdutoImpl`, adiciona e remove quantidades e exibe informações sobre o produto.